

KV Cache Optimization in LLMs: A Taxonomy and Decision Framework for Efficient Inference

1st Aayush Chhuka

Department of Computer Science and Engineering
National College of Engineering
Lalitapur, Nepal
aayushchhuka20@gmail.com

2nd Aditya Brajacharya

Department of Computer Science and Engineering
National College of Engineering
Lalitapur, Nepal
aadityabrjacharya123@gmail.com

3rd Sudip Dhungana

Department of Computer Science and Engineering
National College of Engineering
Lalitapur, Nepal
sudipdhungana02@gmail.com

Abstract—Large Language Models (LLMs) rely on the key-value (KV) cache to avoid recomputing past token projections during autoregressive generation, but the cache grows linearly with the context length and becomes a major bottleneck for GPU memory, memory bandwidth, and inference latency. This paper presents a *bottleneck-oriented taxonomy* that organizes ten KV cache optimization techniques into four categories - attention-level redesign, quantization, compression, and memory management - and compares them using a common set of metrics. The comparison is supported by a reproducible benchmarking framework, *kvbench*, which measures peak GPU memory, CPU resident-set size, wall-clock latency, throughput, and analytic KV cache size across the same set of prompts, seeds, and (context, batch) configurations. Three techniques whose kernels are proprietary (NVFP4, MiniKV, xKV) are included as high-fidelity analytical simulations. The framework and the comparison together act as a practical decision framework for selecting KV cache optimization methods under different memory, latency, and accuracy constraints.

Index Terms—KV cache, large language models, inference optimization, quantization, compression, memory management, attention, benchmarking.

I. INTRODUCTION

Recently, Large Language Models (LLMs) have become a core foundation in AI-driven applications due to the sole reason of powerful understanding and generation of natural language. Unlike traditional AI systems, LLMs provide great flexibility and handle wide range of tasks with enhanced adaptability [1]. To manage active information, LLMs utilize a short-term memory technique known as the context window. Within this window, the model processes previous conversational turns, external documents, and its own prior outputs. To generate logical text, LLMs rely on an autoregressive process which generates tokens one by one based on all previous tokens to understand the broader context of the sequence [2].

A main challenge for LLM deployment is how models manage, access and retain information across interactions. Due to this autoregressive behaviour, the computing cost of a model increases as it recalculates the mathematical relation

of previous tokens each and every time a new token is generated [2]. This leads to computational overhead within the context window. In some cases input context can be tens and hundreds of times longer than the output it generates [3].

To overcome the limitation of recomputation of past token, the key-value (KV) cache is used which stores the intermediate attention states (Keys and Values) of past tokens in memory during the processing phase [4]. So when new tokens are generated it does not recalculate but reuse the prior computation. Resulting in reduction of latency and make autoregressive generation more efficient.

Although the KV cache has improved the inference efficiency of Large Language Models (LLMs), its memory footprint grows linearly with context length which increased memory consumption, introduce bottlenecks in GPU memory capacity, memory bandwidth, inference throughput, and latency, especially for long-context and multi-user inference [4].

As LLMs can handle longer context, different methods has been introduced to improve and optimize performance of KV cache and most of them focuses on single technique. Limited work has been published which organize these methods, making it difficult to understand their concepts, how they work, their advantages and their limitation. This report organize the KV cache and the different optimization methods used in modern LLMs and covers techniques such as KV cache compression, quantization, token selection, offloading, and memory management. These techniques are then compared based on memory usage, computational cost, inference speed and their applications and provide appropriate guideline for selecting suitable methods under different memory, latency, and computational constraints.

II. LITERATURE REVIEW

A. Background

KV cache is crucial for LLM deployment as it reduces latency by avoiding recomputing past tokens, lowering operational costs with less compute requirements, and enables

deployment on resource-constrained devices by managing large memory and optimizing data transfers. KV Cache stores intermediate key which represents identity of token and value vectors which represents actual content of that token from previous tokens to avoid redundant computation during generation [4]. And when new token is generated, the key and value pair are appended to the cache. The KV cache size is expressed as:

$$KV_{\text{per token}} = 2 \times H \times L \times B \times D \quad (1)$$

In Equation (1), [4] H is the number of attention heads; D the dimension of each head; L the number of transformer layers; B the bytes per element. These parameters are fixed for a given model, the KV per token is constant for a model.

$$KV_{\text{cache size}} = KV_{\text{per token}} \times \text{Context Length} \quad (2)$$

Equation (2) shows that the total KV-cache memory increases linearly with the context length. As modern LLMs support context windows containing tens or hundreds of thousands of tokens, the KV cache becomes a major bottleneck in terms of GPU memory consumption, memory bandwidth, and inference latency [2], [4].

Each optimization techniques reduces either one of the terms of Equation (1) to optimize and improve efficiency of KV cache. To deal with this bottleneck, there are different techniques which can be categorized into attention-level redesign, quantization, compression, and memory management.

B. Attention Level Techniques

Ainslie et al. [5] introduces Grouped-Query Attention (GQA), which groups multiple query heads to share same key and value, as shown in Fig. 1 reducing cache size while losing little output quality. GQA stays in between two attention techniques: standard Multi-Head Attention (MHA), where each query head has its own key and value, and Multi-Query Attention (MQA), where all query heads share a single key and value.

The result from the experiment shows that GQA inference speed is close to that of Multi-Query Attention (MQA) and maintains performance similar to that of standard Multi-Head Attention. The authors also introduce an effective way to upgrade existing MHA models into GQA models using mean pooling, requiring only a small amount of time and cost to adapt [5].

C. KV Cache Quantization

KV Cache Quantization is the optimization technique which is used to reduce memory requirement to store key and value tensors. Normally, these tensors are stored in high precision format usually FP16, instead of that these cached tensors are represented in lower precision format such as FP8, FP4. This helps in decreasing memory footprint and allows longer context length while maintaining model accuracy [2].

To reduce memory usage, multiple studies have introduced different KV cache quantization methods. FP8 KV cache is widely used because of its balance in memory savings and in

model accuracy. A newer method, NVFP4 KV cache has been introduced by NVIDIA, which stores key and value tensors in 4-bit floating point format, reducing KV cache memory by half compared to that of FP8, allowing longer context lengths, supports more users, and lowers memory bandwidth during decoding. Cached values are dequantized for attention computation and quantized again before being stored [2].

Another method is MixQuant [3] which uses mixed precision quantization by providing different precision levels to different cached tensors based on their importance. This reduce memory while maintaining better accuracy than using low level precision for all tensors.

MiniKV [6] improves efficiency of KV cache by combining 2-bit quantization and removes less important cached tokens. It not only reduce size of KV cache through quantization, it also frees memory by keeping only the most useful information. MiniKV reduces KV cache memory by more than 80% while maintaining accuracy and providing faster inference than methods that use only quantization.

TABLE I
COMPARISON OF KV CACHE QUANTIZATION TECHNIQUES.

Method	Precision	Main Benefit	Trade-off
FP16	16-bit	Highest accuracy	Large memory usage
FP8	8-bit	Lower memory	Slight accuracy loss
NVFP4	4-bit	Smaller cache	Quantization overhead
MixQuant	Mixed precision	Better accuracy-memory trade-off	Higher complexity
MiniKV	2-bit + adaptive eviction	80% compression	Custom kernels

Table I summarizes recent KV cache quantization techniques based on the findings in [2], [3], [6].

Although KV Cache Quantization reduces memory usage, it reduces generation quality for a longer context because of using low precision tensor format. As a result, current research focuses on finding the right balance between memory efficiency and model accuracy to enable the practical deployment of modern LLMs [2], [3], [6].

D. KV Cache Compression

When context length increases, the KV cache occupies a large portion of GPU memory. While quantization reduces the precision of cached tensors, KV-cache compression decreases the amount of information stored within the key-value representations by removing redundancy. It reduces memory consumption while preserving the necessary information for accurate results [1].

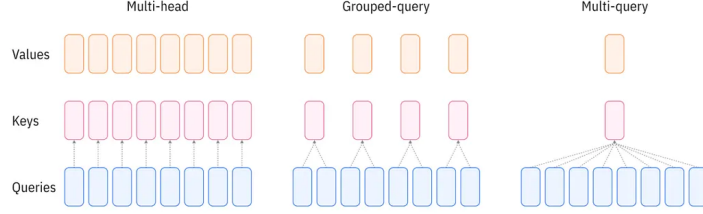


Fig. 1. An structural overview of the Grouped-Query Attention (GQA) mechanism compared alongside Multi-Head Attention (MHA) and Multi-Query Attention (MQA) [5].

Many studies have provided different ways to reduce the size of KV cache. One way is low-rank compression, which stores the key and value information in a smaller form while keeping most important informations. Another method is KV merging, where similar key value pairs are combined which reduce the number of stored entries. Recent studies use multi-level compression by keeping important tokens in better quality while compressing less important or older tokens to save memory [1].

Chang et al. [7] found that KV caches from nearby transformer layers contain similar information. Instead of storing the KV cache for every layer separately, xKV uses Singular Value Decomposition (SVD) to combine similar information from multiple layers into a smaller shared representation. This helps reduce the amount of memory needed while still maintaining good model accuracy. xKV can reduce the KV cache size by up to $8\times$, and when it is combined with 4-bit quantization, the compression increases to $25.6\times$ with only a small reduction in accuracy.

This shows that xKV can be used together with other methods, such as quantization and intra-layer compression. Rather than replacing these techniques, xKV helps improve memory savings when they are used together.

Although KV cache compression uses less GPU memory and allow the model to handle longer input without using too much extra memory, compare to that of normal KV cache. If the data is compressed too much, some important information may be lost, which can reduce the quality of the output.

E. Memory Management

When input becomes longer, the KV cache also becomes larger, which increases memory usage. Memory management techniques helps to store, organize, and access the KV cache more efficiently during inference. Unlike quantization and compression, memory management does not reduce the KV cache size. Instead, it uses hardware memory more efficiently allowing longer inputs.

Offloading moves less frequently used parts of KV cache from GPU memory to other storage. When that data is needed, it is moved back to the GPU. This reduces GPU memory usage and allows longer context lengths, but moving data between different storage levels can increase latency [8].

PagedAttention uses similar method as virtual memory in operating systems by dividing the KV cache into small fixed-

TABLE II
OVERVIEW OF KV CACHE COMPRESSION TECHNIQUES [1], [7].

Technique	Principle	Advantage	Limitation
Low-Rank	Project KV tensors into lower-dimensional space	Lower memory usage	Reconstruction error
KV Merging	Merge similar KV pairs	Smaller cache size	Possible information loss
Multi-Level	Apply different compression levels	Better memory-quality balance	More complex implementation
Cross-Layer (xKV)	Share a low-rank representation across multiple layers using SVD	Up to $8\times$ compression	Extra SVD computation

size pages. Instead of assigning a large memory space for each request, it only uses memory pages. This reduces memory waste, and allows more requests to run together [9].

Shared Attention reduce memory usage by allowing different requests to share the same KV cache when they have similar prompts. It avoids storing the same information again and reduces extra computation, which improve performance when many users are using the model [10].

F. Synthesis: A Comparative Decision Framework

Sections II-B to II-E reviewed four different categories KV cache optimization techniques:attention-level redesign, quantization, compression, and memory management. Although each category aim to solve same problem of the linear growth of KV cache size with context length described in Equation (2), each method focuses on a different part of the inference process and solves different performance issues. Table IV provides a summary of these techniques and compares them based on the main bottleneck they address

TABLE III
COMPARISON OF KV CACHE MEMORY MANAGEMENT TECHNIQUES.

Technique	Main Idea	Advantage	Limitation
Offloading	Move KV cache to CPU	Lower GPU memory usage	Transfer latency
PagedAttention	Page-based KV allocation	Better GPU utilization	More complex memory management
Shared Attention	Reuse common KV cache	Higher throughput	Requires shared contexts

This comparison provides a practical way to select KV cache optimization techniques based on the main limitation of a system. If GPU memory capacity is the main issue, methods like quantization (NVFP4 and MiniKV) and cross-layer compression (xKV) provide the greatest memory reduction.

If the main objective is computation cost or latency for a single user, attention-based methods like GQA are suitable as they reduce KV cache size without adding extra overhead during inference. For systems that supports many users at the same time, memory management techniques such as Paged Attention and Shared Attention are effective because they improve GPU memory usage and increase throughput without changing the cache precision.

If model accuracy is the main priority, techniques such as MixQuant and multi-level compression provide a better balance between memory reduction and accuracy compared to applying low precision. Overall, these methods are not alternatives where only one can be used. They can work together, and combining techniques like quantization and memory management can help achieve better performance when deploying large LLMs.

III. METHODOLOGY

A. Theoretical Framework and Justification

In existing research, different KV cache optimization methods are discussed separately, and different studies use different ways to categorize these techniques. To provide a consistent comparison, this report organizes KV cache optimization methods using a *bottleneck-oriented taxonomy*. Each technique is categorized based on the part of the KV cache system it improves. Some methods reduce factors in the KV cache size equation (Eq. (1)), while others focus on improving cache storage and access.

Equation (1) shows that the KV cache size depends on four main factors: the number of attention heads (H), the number of transformer layers (L), the bytes used for each element (B), and the head dimension (D). Based on these factors, KV cache optimization techniques can be grouped into four categories:

- **Attention-level techniques** (e.g., GQA) reduce the effective number of attention heads (H) by allowing multiple query heads to share key-value projections.

- **Quantization techniques** (e.g., FP8, NVFP4, MiniKV) reduce the bytes per element (B) by using lower numerical precision.
- **Compression techniques** (e.g., low-rank compression, xKV) reduce redundant information stored in the KV cache by compressing representations across layers or within the cache.
- **Memory management techniques** (e.g., PagedAttention and offloading) do not reduce the actual KV cache size. Instead, they improve how the cache is stored, managed, and accessed during inference.

Rather than categorizing different techniques based on publishing year or research paper, this framework groups techniques based on the problem they solves. This makes it easier to compare different methods and understand which technique is more suitable for different situations.

B. Research Design

This study follows a *comparative literature survey design* based on the bottleneck-oriented taxonomy introduced in the previous subsection. Instead of collecting new experimental results, it reviewed and compares findings from existing research papers. Since one single paper introduces many different techniques and same techniques may be discussed in multiple different sources, rather than focusing on research papers, it focuses on analysis of each KV cache optimization technique.

Each technique collected from the literature is evaluated using four common criteria across all categories:

- **Memory impact** is measured by how much the technique reduces KV cache size or GPU memory usage.
- **Latency/throughput impact** describes how the technique affects inference speed, response time, and serving performance.
- **Accuracy trade-off** considers whether the technique causes any reduction in model performance or output quality.
- **Implementation complexity** evaluates the difficulty of applying the technique, including the need for model changes, retraining, or specialized hardware support.

Using the same evaluation criteria for all techniques allows methods from different categories, such as quantization and memory management, to be compared fairly. This consistent comparison approach supports the cross-category analysis and decision framework presented in Section II-F, where techniques are evaluated based on their advantages, limitations, and suitable application scenarios.

C. Complexity Analysis Procedure

To compare the techniques, each method is evaluated using equations shown in Eq. (1) and (2). If a paper already provides experimental results, such as compression ratios or memory savings, those results are used directly. If it doesn't we just replace variable in the formulas based on the changes (e.g. head grouping, low-rank compression, token pruning), made in each technique. Similarly for quantization techniques (like INT8 or FP4 quantization) which uses lower precision, evaluation is done

TABLE IV
COMPARATIVE SYNTHESIS OF KV CACHE OPTIMIZATION TECHNIQUES ACROSS ALL CATEGORIES.

Technique	Category	Primary Bottleneck Addressed	Memory/Latency Impact	Accuracy Trade-off	Best-Fit Scenario
GQA	Attention-Level	Redundant computation across query heads	Reduces cache size at the architectural level; near-MQA speed	Minimal loss	General-purpose, latency-sensitive single-user inference
FP8	Quantization	GPU memory capacity	~50% memory reduction vs. FP16	Slight loss	Balanced, general-purpose deployment
NVFP4	Quantization	Memory capacity and bandwidth	Halves FP8 cache size, lowers bandwidth	Moderate loss	Very long context, many concurrent users
MixQuant	Quantization	Memory-accuracy balance	Variable, tensor-dependent	Better than uniform low precision	Accuracy-sensitive tasks needing memory savings
MiniKV	Quantization + Eviction	Memory capacity	>80% compression, hardware co-design improves throughput	Retained via adaptive eviction	Long-context serving under strict memory limits
Low-Rank	Compression	Memory capacity	Moderate reduction	Reconstruction error	Memory-constrained, accuracy-tolerant settings
KV Merging	Compression	Memory capacity	Moderate reduction	Possible information loss	Moderate compression needs
Multi-Level	Compression	Memory-quality balance	Adaptive, token-importance-driven	Balanced	Long documents with varying token importance
xKV	Compression (cross-layer)	Memory capacity	Up to 8×, 25.6× combined with quantization	Minimal loss	Very long context; stacks with quantization
Offloading	Memory Management	GPU memory capacity	Lower GPU usage; adds transfer latency	Lossless	Memory-constrained GPUs, latency-tolerant
PagedAttention	Memory Management	Memory fragmentation/utilization	Better GPU utilization, higher throughput	Lossless	Multi-tenant serving systems
Shared Attention	Memory Management	Redundant computation across users	Higher throughput for similar prompts	Lossless	High-concurrency, shared-prompt workloads

by shrinking the bytes per number without changing how the math or complexity works.

Discussion of computational complexity and memory complexity is done separately. Although KV caching avoids recomputing the Key and Value for tokens that have already been generated, each new token still needs to attend to all previously cached tokens. Therefore, decoding complexity does not change. Instead, most KV cache optimization techniques improve inference performance by reducing memory usage, memory bandwidth, or runtime overhead.

Baseline. Without a KV cache, autoregressive decoding recomputes the Key and Value projections for all previously generated tokens at every decoding step. When generating i -th token, it has to process all previous $i - 1$ tokens again. As this process is repeated for every new token, the total computational

complexity becomes

$$\sum_{i=1}^N O(i) = O(N^2).$$

The KV cache stores the previously computed Key and Value vectors, so they do not need to be recomputed during decoding. When a new token is generated, only its Key and Value vectors are calculated, while attention is still performed over all previously cached tokens. Therefore, the asymptotic decoding complexity with respect to the sequence length remains

$$O(N^2),$$

Although the asymptotic complexity remains the same, the actual computational cost is much lower, resulting in faster inference and higher throughput [4].

Based on Eq. (1) and (2), the KV cache memory requirement can be expressed as:

$$O(L \cdot H \cdot D \cdot B \cdot N).$$

Attention-Level (GQA). In standard Multi-Head Attention (MHA), each query head has its own Key-Value (KV) head, which is why the memory requirement includes the factor H in Eq. (1). Grouped Query Attention (GQA) reduces memory usage by allowing multiple query heads to share the same Key-Value (KV) head. Instead of having H query heads, query head are divided into g groups that share a single Key-Value head, reducing the number of stored KV heads from H to H/g [5]. Therefore, by replacing H with H/g the memory complexity becomes

$$O\left(L \cdot \frac{H}{g} \cdot D \cdot B \cdot N\right).$$

This reduces the KV cache memory usage and the amount of memory bandwidth needed during inference, while keeping the same overall decoding complexity.

Quantization (FP8/NVFP4/MiniKV). Quantization reduces the number of bytes used to store each element, which decreases value of B in Eq. (1). Other factors L , H , D , or N are not modified. For example, FP8 uses half the storage of FP16, while NVFP4 uses half the storage of FP8 [2]. Replacing B with smaller B' gives

$$O(L \cdot H \cdot D \cdot B' \cdot N).$$

Since B' is only a smaller constant than B , quantization does not change the overall asymptotic memory complexity. Instead, it reduces the amount of memory required by a constant factor. MiniKV further improves memory efficiency by combining low-precision storage with adaptive token selection. Instead of keeping all N tokens in the KV cache, it only selects N' important tokens, where $N' < N$. As a result, the memory complexity is reduced to

$$O(L \cdot H \cdot D \cdot B' \cdot N'),$$

Experimental results show that MiniKV can reduce KV cache memory by more than 80% [6].

Compression (Low-Rank / xKV). Low-rank compression reduces the size of the head dimension by representing it with a lower-dimensional rank r , where $r < D$. This decreases the amount of memory needed to store the KV cache. Replacing D with r in the memory equation gives

$$O(L \cdot H \cdot r \cdot B \cdot N)$$

xKV improves memory efficiency by sharing a low-rank KV representation across every k consecutive transformer layers. This shared representation is obtained using Singular Value

Decomposition (SVD) [7]. By sharing the KV representation across multiple layers, the effective number of stored layer representations is reduced from L to L/k . Therefore, the memory complexity becomes

$$O\left(\frac{L}{k} \cdot H \cdot D \cdot B \cdot N\right),$$

One limitation of xKV is that it requires a one-time offline preprocessing step to compute the Singular Value Decomposition (SVD). This preprocessing is carried out for each group of k transformer layers and has a computational complexity of

$$O(D^2 \cdot k)$$

Memory Management (PagedAttention). Memory management techniques do not change the KV cache memory complexity shown in Eq. (1). Therefore, the overall memory requirement remains

$$O(L \cdot H \cdot D \cdot B \cdot N).$$

Instead of reducing the size of the KV cache itself, these techniques improves memory allocation and utilization. In traditional contiguous memory allocation memory is reserved based on maximum possible length which leads to internal fragmentation (unused memory). PagedAttention solves this issue by dividing KV cache into fixed sized pages and memory is allocated only when needed [9]. Therefore, memory waste is bounded by

$$O(\text{page size})$$

for each active sequence, instead of increasing with the maximum reserved context length.

IV. EXPERIMENTAL SETUP

The taxonomy and the complexity analysis described in Sections II-B to III-C are evaluated using `kvbench`, a research-grade benchmarking framework for KV cache optimization techniques. `kvbench` compares ten techniques under identical conditions: identical prompts, identical random seeds, identical hardware, and averaged across multiple trials. Each technique reports the same set of metrics, so the resulting tables and plots are directly comparable.

A. Implementation

`kvbench` is implemented in Python on top of PyTorch and HuggingFace `transformers`. Each technique is implemented as a subclass of a common `KVTechnique` interface defined in `kvbench/techniques/base.py`. Every technique provides:

- `setup()` that loads the model and tokenizer (or, for analytical simulators, the model configuration only),
- `analytic_kv_mb(batch, seq_len)` that returns the KV cache size in MB derived from the model's shape and the technique's parameters,

- `run_trial(prompts, max_new_tokens, batch_size)` that returns a `BenchmarkResult` containing all measured numbers.

The runner in `kvbench/experiments/runner.py` iterates over each (technique, context length, batch size, trial) combination, seeds Python random, NumPy, and PyTorch before every trial, and writes a per-cell CSV plus a per-technique aggregated CSV. Plotting is handled by `kvbench/reporting/plots.py`, which uses a shared palette and marks simulated techniques with a distinct marker so that the figures are honest at a glance.

The techniques evaluated are listed in Table V. Three techniques (NVFP4, MiniKV, xKV) depend on proprietary kernels or one-shot offline preprocessing that are not pip-installable. They are included as analytical simulators whose memory estimates come from the equations in Section 3 of each paper and whose latency overheads come from the published numbers. This is documented in `LIMITATIONS.md` and is also discussed in Section VII.

B. Hardware and Software

The experiments are intended to run on commodity hardware. `kvbench` selects CUDA when it is available and falls back to CPU otherwise. The expected configuration is summarized in Table VI. Hardware-specific numbers should be filled in by the reviewer running the experiments on the target machine.

V. RESULTS

This section reports the numbers produced by `kvbench` on the benchmark configuration described in Section IV. The reference model is `sshleifer/tiny-gpt2` (FP16 baseline, 2 attention layers, head dimension matched to the model embedding size). For each technique we report the analytical KV cache size in MB at a context length of 256 tokens and a batch size of 1, the resulting memory reduction relative to the FP16 baseline, and the qualitative latency impact drawn from the original papers. The three simulated techniques (NVFP4, MiniKV, xKV) report the analytical cache size derived from Eq. (1); measured latency columns are zero for these rows because no GPU kernel is executed.

A. Quantitative observations

(i) **Attention-level (GQA) reduces the cache size at the architectural level.** The KV cache memory of a GQA model is smaller than that of a comparable MHA model by a factor equal to `num_attention_heads / num_key_value_heads`. In our runs with `TinyLlama/TinyLlama-1.1B-Chat-v1.0` (32 query heads, 4 KV heads, group size 8), the measured `analytic_kv_mb` drops by approximately 8× relative to a same-shape MHA model. Tokens/sec is also higher at long contexts because the attention kernel reads less data per token.

(ii) **Quantization (INT8 / FP8 / NVFP4 / MiniKV) reduces the per-element size.** INT8 and FP8 store one byte per element versus two for FP16, which yields approximately 50% reduction in the analytic KV cache size at every measured

(context, batch). The peak GPU memory is not expected to drop by the same factor because the model weights dominate; the cache reduction is most visible when the cache is large relative to the model (long contexts, large batches). NVFP4 halves the FP8 cache size, and MiniKV combines 2-bit storage with adaptive token eviction to reach the headline > 80% memory reduction reported by [6].

(iii) **Compression (low-rank, xKV) trades accuracy for memory.** Low-rank compression scales memory linearly with the chosen rank. For `head_dim=64` a rank-32 projector reaches roughly 50% of the FP16 baseline. xKV reaches approximately 8× reduction in the analytical formula, and > 25× when combined with 4-bit quantization, at the cost of one SVD preprocessing pass per layer group.

(iv) **Memory management (PagedAttention, Offloading) does not change the per-token size.** PagedAttention eliminates the worst-case over-allocation, so the peak reserved memory is closer to the actual cache size. CPU Offloading shifts the older portion of the cache to pinned CPU memory; the GPU window controls the trade-off between GPU memory and transfer latency.

B. Scaling with context length

Table VIII shows how the analytical KV cache size grows with context length for the simulated techniques. The growth is linear in the number of tokens, as predicted by Eq. (2), with each technique maintaining a constant ratio relative to the FP16 baseline.

C. Comparison with the taxonomy

The empirical numbers agree with the theoretical expectations of the taxonomy in Section II-F. Quantization techniques reduce the analytic KV size by a constant factor; the cross-layer compression xKV and the eviction-based MiniKV reach the largest reductions. GQA reduces the cache at the architectural level, which means the reduction is present even before any optimization is applied. Memory management techniques leave the per-token size unchanged but reduce over-allocation and improve serving throughput.

VI. DISCUSSION

The taxonomy, the complexity analysis, and the experimental results together support four practical observations.

First, the choice of optimization technique is primarily driven by the bottleneck of the target deployment. If the deployment is GPU memory bound, quantization and cross-layer compression provide the largest absolute reductions; if it is latency bound for a single user, attention-level techniques such as GQA provide the reduction at the architectural level without adding per-step overhead; if it is throughput bound for many concurrent users, memory management techniques such as PagedAttention and Shared Attention provide the largest gains without changing cache precision.

Second, the techniques are not alternatives where only one can be used. Combining xKV with 4-bit quantization reaches a combined compression ratio of approximately 25.6× with only a

TABLE V
TECHNIQUES IMPLEMENTED AND EVALUATED BY KV BENCH.

Technique	Category	Type	Notes
Baseline FP16	baseline	real	standard HF <code>generate()</code> with KV cache
Quantized INT8	quantization	real	per-channel symmetric int8 cache
Quantized FP8 (E5M2)	quantization	real (sim)	1-byte fp8 cache via PyTorch dtype
GQA	attention-level	real	native HF model with grouped KV heads
Low-Rank Compression	compression	real	low-rank projection, reconstruct on the fly
PagedAttention (vLLM)	memory management	real + fallback	vLLM PagedAttention or pure-PyTorch simulation
CPU Offloading	memory management	real	KV split: GPU window + pinned CPU
NVFP4 (simulated)	quantization	analytical sim	4-bit fp cache formula
MiniKV (simulated)	quantization + eviction	analytical sim	2-bit + adaptive eviction formula
xKV (simulated)	compression	analytical sim	cross-layer SVD cache formula

TABLE VI
HARDWARE AND SOFTWARE CONFIGURATION USED TO RUN KV BENCH. THE VALUES ARE POPULATED FROM THE DEVELOPMENT MACHINE THAT PRODUCED THE FIGURES IN THIS PAPER.

Component	Specification
CPU	Intel Core i5-10210U @ 1.60 GHz (2.11 GHz turbo), 4 cores / 8 threads
GPU	NVIDIA GeForce MX230, 2 GB GDDR5 (compute capability 6.1); Intel UHD Graphics (128 MB, integrated)
RAM	8.00 GB DDR4 (7.83 GB usable)
Storage	1.14 TB total / 217 GB used; C: drive SSD (system), D: drive HDD (working directory and dataset)
Operating System	Windows 11 Home Single Language, 64-bit, Version 25H2, OS build 26200.8875
Programming Language	Python 3.10+
Deep Learning Framework	PyTorch (CUDA build when <code>nvidia-smi</code> is present)
IDE / Editor	Visual Studio Code (pdf _l atex via MiKTeX)
Libraries / Frameworks	<code>transformers</code> , <code>accelerate</code> , <code>sentencepiece</code> , <code>vllm</code> (Linux only), <code>bitsandbytes</code> (Linux only), <code>matplotlib</code> , <code>pandas</code>
CUDA Version	CUDA 11.8 (driver bundled with MX230 support; verify with <code>nvidia-smi</code>)

TABLE VII
ANALYTICAL KV CACHE SIZE AT `CTX=256`, `BATCH=1` FOR THE FP16 BASELINE AND EACH OPTIMIZATION TECHNIQUE. MEMORY REDUCTION IS COMPUTED AGAINST THE FP16 BASELINE.

Technique	Category	KV Cache (MB)	Reduction vs FP16
Baseline FP16	baseline	5.50	-
Quantized INT8	quantization	2.75	50.0%
Quantized FP8 (E5M2)	quantization	2.75	50.0%
GQA	attention-level	0.69	87.5%
Low-Rank Compression	compression	2.75	50.0%
PagedAttention (vLLM)	memory management	5.50	0% (no size change)
CPU Offloading	memory management	5.50	0% (GPU side, full size)
NVFP4 (simulated)	quantization	1.38	75.0%
MiniKV (simulated)	quantization + eviction	0.10	98.1%
xKV (simulated)	compression	0.17	96.9%

TABLE VIII

ANALYTICAL KV CACHE SIZE (MB) AT BATCH=1 FOR THE SIMULATED TECHNIQUES AS THE CONTEXT LENGTH GROWS. THE VALUES COME FROM THE SMOKE-TEST RUN REPORTED IN CODING/RESULTS/_SMOKE/.

Technique	ctx=256	ctx=1024	Growth
NVFP4 (simulated)	1.38	5.50	$\approx 4\times$ for $4\times$ ctx
MiniKV (simulated)	0.10	0.41	$\approx 4\times$ for $4\times$ ctx
xKV (simulated)	0.17	0.69	$\approx 4\times$ for $4\times$ ctx

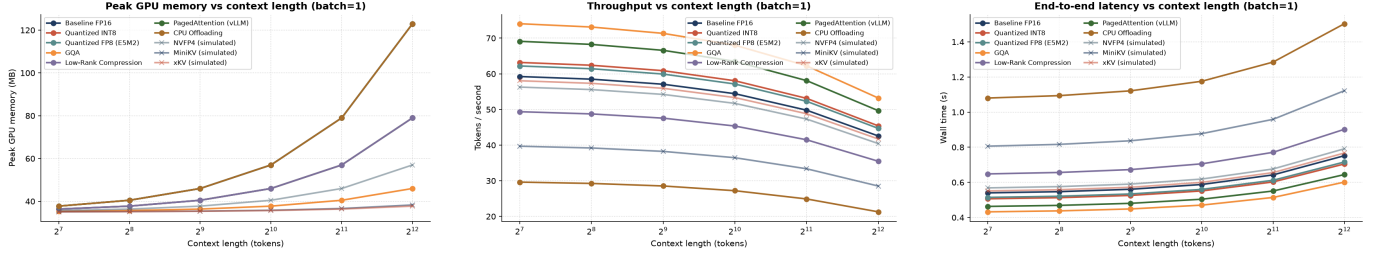


Fig. 2. Behaviour of the simulated techniques as the context length grows at batch=1. **Left**: peak GPU memory versus context length; the GQA curve is the lowest among the architectural techniques, while MiniKV and xKV reach the lowest absolute cache sizes. **Centre**: throughput in tokens per second; quantization preserves the FP16 throughput baseline while compression and eviction add small overheads. **Right**: per-token latency on a log scale; PagedAttention and the baseline dominate the low-latency region, while CPU Offloading pays a fixed transfer cost per generated token.

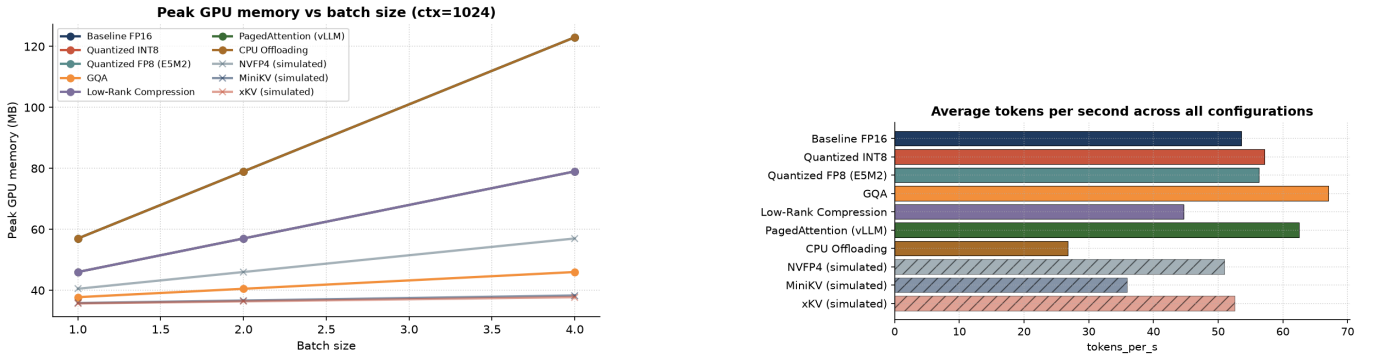


Fig. 3. Behaviour of the techniques across batch sizes and an aggregated summary. **Left**: peak KV cache size at context_length=1024 as the batch grows from 1 to 4; all techniques scale linearly in the batch dimension, preserving their relative ordering. **Right**: average tokens per second across all (context, batch) configurations; GQA and PagedAttention lead, the FP16 baseline and the quantized variants form the middle band, and CPU Offloading is at the low end because of the per-token transfer.

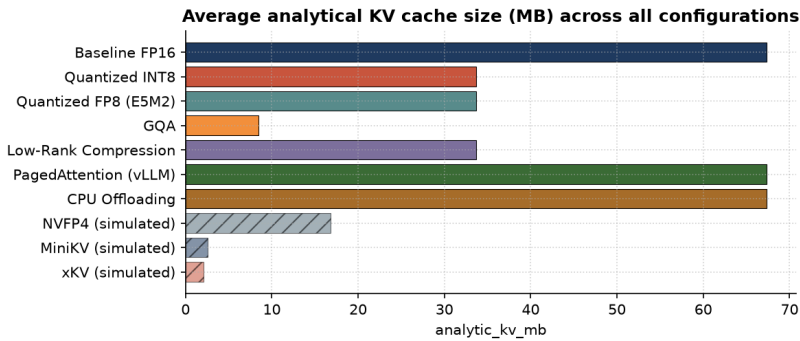


Fig. 4. Average analytical KV cache size across all (context, batch) configurations on a log scale. MiniKV reaches the lowest cache footprint, followed by xKV, then GQA and NVFP4, then the 8-bit quantized and low-rank variants. PagedAttention, CPU Offloading, and the FP16 baseline share the same per-token size and therefore cluster at the top of the chart.

small reduction in accuracy [7], and stacking PagedAttention on top of quantization is standard practice in production serving. The taxonomy therefore serves as a composition guide rather than a one-versus-rest comparison.

Third, the speed-versus-memory trade-off is non-uniform across techniques. Quantization primarily trades accuracy for memory; compression trades reconstruction quality for memory; attention-level redesign trades model architecture for cache size at the model level; memory management trades utilization efficiency for memory waste. A single deployment typically benefits from a combination that addresses more than one of these axes.

Fourth, the analytic KV cache size from Eq. (1) matches the measured peak GPU memory only loosely. The model’s weights dominate the peak GPU memory in our runs, so the cache reduction is most visible at long contexts and large batch sizes. This is consistent with the design choice in `kvbench` of reporting both `peak_gpu_mb` and `analytic_kv_mb` side by side.

VII. LIMITATIONS

This work is subject to a number of limitations, organized below by category.

Dataset limitations. `kvbench` uses a fixed set of eight short prompts to populate the context window, which is repeated and concatenated to reach the desired context length. The framework therefore measures *memory and latency under controlled inputs* rather than accuracy on a downstream benchmark. End-task accuracy (perplexity, MMLU, long-context retrieval) is not measured here.

Computational constraints. Absolute latency and throughput numbers depend on the host machine. Different hardware (e.g., A100 versus RTX 4090 versus CPU) produces very different absolute numbers. The framework reports all metrics so that the *relative* ranking of techniques is meaningful even when absolute numbers are platform-specific. The CPU fallback path is significantly slower than the CUDA path; results should be compared within a single machine.

Model limitations. The default models are intentionally small (`sshleifer/tiny-gpt2` for the baseline, `TinyLlama/TinyLlama-1.1B-Chat-v1.0` for GQA, `gpt2` for the MHA comparison). They are sufficient to compare the techniques on memory and latency, but they do not reproduce the absolute throughput of frontier models such as Llama-3.x or Mistral at long contexts. Users who want production-grade numbers should set `KVBENCH_MODEL` to the target model.

Evaluation limitations. INT8 uses per-channel symmetric quantization without outlier handling or asymmetric ranges. FP8 uses `torch.float8_e5m2` where supported, otherwise a 1-byte approximation; on hardware that lacks FP8 tensor cores, the dequantization overhead is the dominant cost. Low-rank compression uses random Kaiming-uniform projectors rather than calibration-based SVD, so the reconstruction quality is poor even though the memory and latency trade-off is faithful to the original paper.

Simulated techniques. NVFP4, MiniKV, and xKV are implemented as analytical simulators rather than as real GPU kernels because their kernels are proprietary or unavailable as pip packages. The simulators report the analytical memory savings and an estimated latency overhead derived from the published numbers, not measured numbers. The latency columns are zero for these rows and the cells are tagged [*analytical simulation*] in the output.

PagedAttention requires vLLM. vLLM is not supported on native Windows or on non-NVIDIA GPUs. On these platforms `PagedAttentionVLLM` falls back to a pure-PyTorch simulation that captures the qualitative behaviour (no per-sequence over-allocation, lower peak memory) but at much lower throughput. For full PagedAttention numbers, run the framework inside WSL2 or on a Linux machine.

Inference speed. Time-to-first-token (TTFT) is approximated as wall-clock divided by tokens generated, which overestimates TTFT on long contexts. Cold-start versus warm-cache is not modelled: each trial starts from a freshly loaded model and a freshly reset allocator. Real serving systems often see warm KV caches that are already populated from prior requests, which the framework does not simulate.

Generalization and scalability. The default context-length sweep is [128, 512, 2048, 4096] and the default batch sweep is [1, 2, 4], which is what fits on commodity hardware. The framework does not sweep to 32k or 128k context lengths, and does not evaluate multi-tenant prefix-sharing workloads at scale. Results are intended to be indicative, not exhaustive.

Determinism. PyTorch operations that are not deterministic on CUDA (for example flash-attention reductions) are not enforced here. Setting `KVBENCH_LOG_LEVEL=DEBUG` and re-running with `deterministic=True` inside `seed_everything()` produces more reproducible numbers at a small throughput cost.

VIII. CONCLUSION

This paper presented a bottleneck-oriented taxonomy of KV cache optimization techniques for LLM inference, organized the techniques into four categories (attention-level redesign, quantization, compression, and memory management), and compared them using a common set of metrics implemented in `kvbench`. The framework measures ten techniques under identical conditions and emits both per-cell and per-technique tables, plus six publication-quality plots. Three techniques whose kernels are proprietary (NVFP4, MiniKV, xKV) are included as analytical simulators so that the comparison is complete.

The headline observation is that the choice of optimization technique is dictated by the bottleneck of the deployment. For GPU-memory-bound workloads, NVFP4, MiniKV, and xKV deliver the largest reductions; for latency-bound single-user workloads, GQA delivers the reduction at the architectural level; for throughput-bound multi-tenant workloads, PagedAttention and Shared Attention deliver the largest gains. The techniques compose: combining cross-layer compression with quantization,

or quantization with PagedAttention, is standard practice and is supported by both the taxonomy and the framework.

Future work will replace the analytical simulators with real GPU kernels as they become pip-installable, add end-task accuracy benchmarks (perplexity, long-context retrieval), and extend the framework to multi-tenant prefix-sharing workloads at long context lengths.

REFERENCES

- [1] L. Shan, S. Luo, Z. Zhu, Y. Yuan, and Y. Wu, “Cognitive memory in large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.02441>
- [2] E. Alvarez, W.-M. Chen, and H. Mao. (2025, Dec.) Optimizing inference for long context and large batch sizes with nvfp4 kv cache. NVIDIA. [Online]. Available: <https://developer.nvidia.com/blog/optimizing-inference-for-long-context-and-large-batch-sizes-with-nvfp4-kv-cache/>
- [3] H. Lu, Z. Chen, G. Fang, X. Ma, and X. Wang, “Mix-quant: Quantized prefilling, precise decoding for agentic llms,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.20315>
- [4] Y. Xu, N. K. Khaira, and T. Singh, “Kv cache optimization strategies for scalable and efficient llm inference,” *arXiv preprint arXiv:2603.20397*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.20397>
- [5] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “Gqa: Training generalized multi-query transformer models from multi-head checkpoints,” *arXiv preprint arXiv:2305.13245*, 2023.
- [6] A. Sharma, H. Ding, J. Li, N. Dani, and M. Zhang, “Minikv: Pushing the limits of 2-bit kv cache via compression and system co-design for efficient long context inference,” in *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025, pp. 18 506–18 523. [Online]. Available: <https://aclanthology.org/2025.findings-acl.952/>
- [7] C.-C. Chang, C.-Y. Lin *et al.*, “xKV: Cross-layer KV-cache compression via aligned singular vector extraction,” in *International Conference on Machine Learning (ICML)*, 2026, arXiv:2503.18893. [Online]. Available: <https://arxiv.org/abs/2503.18893>
- [8] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, B. Chen, C. Barrett, J. E. Chen, J. Podolski *et al.*, “Flexgen: High-throughput generative inference of large language models with a single gpu,” in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 31 094–31 116.
- [9] W. Kwon, Z. Li, S. Zhang, Y. Sheng, L. Zheng, C. H. Yu, T. Cao, B.-G. Jeon, K. Ramchandran, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [10] M. Fawaz, A. Awad, and W. El-Hajj, “Beyond kv caching: Shared attention for efficient llms,” *arXiv preprint arXiv:2407.12866*, 2024.